AI GOVERNANCE ENGINE / KNOWLEDGE BASE SAMPLE

Human-readable Knowledge Base document

Sample structure and Engine JSON translation for category authors

Use this sample when writing the human-readable category PDF. The section order follows the A1 / AP-A1 publication pattern. Every controlled section must be generated from schema 2.1.0 JSON. The Engine never reads the PDF; it compiles the JSON into six runtime collections and hash-pins them in a knowledge manifest.

SCHEMA ENGINE PUBLICATION

SCHEMA = required or optional canonical JSON field. ENGINE = compiled runtime key the Engine consumes. PUBLICATION = human-review prose that is not a schema 2.1.0 object unless you encode it in a listed field.

Capability	A1 — Intended purpose and use boundaries
Anti-pattern	AP-A1 — Undefined or elastic intended purpose
Canonical schema	AI Governance schema 2.1.0
Authoring objects	knowledge-authoring/example/A1_v1.0.json and AP-A1_v1.0.json
Release status in sample	DRAFT (structurally complete example, not a production activation)
Engine collections	normativeSources, requirements, controls, antipatterns, tactics, intakeQuestionnaire
Authority	Canonical JSON is authoritative. This PDF is a generated view for authors.

Authority boundary

No model output may raise assurance, clear a hard gate, accept residual risk, define legal compliance or authorize a lifecycle transition. Missing or conflicting evidence remains UNKNOWN. Silence never establishes anti-pattern absence.

SECTION

How authors should use this sample

Produce one capability JSON object and one paired anti-pattern JSON object per category. Validate against schemas/capability.schema.json and schemas/antipattern.schema.json, then render the human PDF from those objects. Compile only after the pair, Tactic Catalog mappings and source register pass. Upload the compiled JSON, not this PDF, to the Engine knowledge manifest.

- Keep IDs stable: A1 / AP-A1, A1-Q1, A1-SC-001, EVD-A1-001, FND-A1-001, TAC-PURPOSE-A1-01.
- Primary questions are exactly three, in order: DEFINITION_AND_INTENT, IMPLEMENTATION_AND_OPERATION, EVIDENCE_AND_EFFECTIVENESS.
- Atomic items must cite required_evidence_ids that exist on the same object.
- APPROVED or FROZEN objects require an approval_record. The sample pair is DRAFT, so approval_record is null.
- Tactic retrieval is owned by the Playbook Primary object / mapping. candidate_tactic_refs must not invent unmapped tactic IDs.

Document skeleton

The human PDF uses a fixed outline so reviewers can compare categories. The Engine does not parse these headings. It consumes the JSON fields named in the SCHEMA column after compile.

Package / release identity

SCHEMA

JSON
id, version, schema_version, release_status, domain, object_type, paired_object_id

ENGINE
Loaded as authoring identity on REQ-/CTRL- entries; release status is a knowledge-snapshot diagnostic

Cognitive framing

SCHEMA ENGINE PUBLICATION

JSON
distinct_claim, failure_mechanism, primary_questions.dimension

ENGINE
failureMechanism is copied onto the runtime anti-pattern. distinct_claim is authoring-only today

1. Governance ownership and boundary

SCHEMA ENGINE

JSON
title, canonical_definition, governance_purpose, applicability, runtime_applicability, runtime_authority, runtime_severity, runtime_signals, lifecycle_stages

ENGINE
requirements.interpretation / governancePurpose; controls.severity, signals, lifecycleStages, applicability

2. Primary questions

SCHEMA ENGINE

JSON
primary_questions[3] with fixed dimensions Q1-Q3

ENGINE
controls.questions. Coverage matrix enumerates each question ID

3. Atomic assessment logic

SCHEMA ENGINE

JSON
atomic_subcriteria (capability) or atomic_tests (anti-pattern)

ENGINE
controls.atomicSubcriteria or antipatterns.atomicTests. Finding lock and coverage use these IDs

4. Evidence requirements

SCHEMA ENGINE

JSON
required_evidence[] {id, title, description, minimum_technical_assurance, required_human_assurance}

ENGINE
controls.requiredEvidence / antipattern requiredEvidence. Evidence class, acceptance conditions and limitations in a Word/PDF are publication-only unless encoded here

5. Evidence discipline

SCHEMA ENGINE

JSON
evidence_rules.sufficiency, evidence_ceiling, false_positive_guards, prohibited_inferences

ENGINE
controls.evidenceRules plus flattened falsePositiveGuards and prohibitedInferences

6. Findings and gate effects

SCHEMA ENGINE

JSON
finding_definitions[], hard_gate_effect, human_decision_authority

ENGINE
findingDefinitions on REQ/CTRL/AP objects; hardGateEffect. Locked findings must cite a published finding ID

7. Tested-absence contract

SCHEMA ENGINE

JSON
absence_test_contract (anti-pattern only)

ENGINE
antipatterns.absenceTestContract. TESTED_ABSENT is not inferred from silence

8. Lifecycle assurance targets

SCHEMA ENGINE

JSON
target_assurance_by_lifecycle_stage[]

ENGINE
minimumTechnicalAssuranceByLifecycle, requiredHumanAssuranceByLifecycle, targetStateByLifecycle

9. Normative source mappings

SCHEMA ENGINE

JSON
normative_source_mappings[] resolved against the source register

ENGINE
requirements.normativeMappings and the normativeSources collection. Links are provenance, not legal conclusions

10. Approved tactic mappings

SCHEMA ENGINE

JSON
candidate_tactic_refs[] plus Playbook Primary object / mapping

ENGINE
Playbook tactics retrieve only after a locked finding carries A1-F5 / AP-A1-AP-F5

11. Runtime decision boundary

PUBLICATION

JSON
Not a schema 2.1.0 object. Encode machine limits in evidence_rules and hard_gate_effect

ENGINE
The Engine already forbids models from raising assurance, clearing gates or issuing formal approval

12. Approval record

SCHEMA

JSON
approval_record {approved_by_role, approved_on, approval_scope, approved_version, supersedes_version}

ENGINE
Required before release_status APPROVED or FROZEN. Snapshot diagnostics expose release status

OBJECT / RUNTIME

A1 — Intended purpose and use boundaries

Version 1.0.0 | DRAFT | CAPABILITY | paired AP-A1

Canonical definition

Intended purpose, users, affected persons, operating context, outputs, permitted and prohibited uses, and foreseeable misuse are precise enough to govern design and evaluation.

1. Governance ownership and boundary

SCHEMA ENGINE

Canonical JSON

governance_purpose, distinct_claim, applicability.default_state, applicability.conditions, runtime_applicability, runtime_authority, runtime_severity, runtime_signals, lifecycle_stages, paired_object_id

What the Engine loads after compile

Compiled onto requirements (interpretation, governancePurpose, humanAuthority, applicability) and controls (severity, signals, lifecycleStages, pairedObjectId). runtime_signals become lexical indicators only; they cannot independently establish IMPLEMENTED.

governance_purpose	Prevent purpose ambiguity and uncontrolled expansion of the solution use boundary.
distinct_claim	The intended purpose and use boundary are explicit, attributable and controlled.
applicability.default_state	APPLICABLE
runtime_applicability	ALWAYS
runtime_authority	INTERNAL_PROCESS
runtime_severity	HIGH
runtime_signals	purpose, accountable-owner, excluded-use
paired_object_id	AP-A1
human_decision_authority	GOVERNANCE

applicability.conditions

- Applies to every assessed AI solution.

lifecycle_stages

QUALIFICATION AND REGISTRATION | DESIGN AND DEVELOPMENT | VERIFICATION AND VALIDATION | DEPLOYMENT | OPERATION AND MONITORING | REVIEW AND EVALUATION | RETIREMENT

Optional publication prose

Exclusions and reassessment-trigger lists in the long-form A1 PDF are reviewer aids. Encode anything the Engine must enforce in applicability.conditions, evidence_rules, hard_gate_effect or finding_definitions.

2. Primary questions

SCHEMA ENGINE

Canonical JSON

primary_questions: exactly three objects {id, dimension, question}. Dimensions are fixed as A1-Q1 DEFINITION_AND_INTENT, A1-Q2 IMPLEMENTATION_AND_OPERATION, A1-Q3 EVIDENCE_AND_EFFECTIVENESS.

What the Engine loads after compile

Copied to controls.questions. Cognitive coverage enumerates each question ID. The Engine assesses the loaded instrument even before evidence rules are published; atomic IDs and finding IDs are what make locked findings precise.

A1-Q1	DEFINITION AND INTENT — Are intended purpose, users, outcomes, permitted uses and prohibited uses explicitly defined?
A1-Q2	IMPLEMENTATION AND OPERATION — Do implementation and operational boundaries enforce the declared intended purpose?
A1-Q3	EVIDENCE AND EFFECTIVENESS — Does current attributable evidence demonstrate that the purpose boundary remains effective?

3. Atomic assessment logic

SCHEMA ENGINE

Canonical JSON

atomic_subcriteria[] {id, question_id, criterion, required_evidence_ids, minimum_technical_assurance, required_human_assurance}. IDs match ^[A-F][1-5]-SC-[0-9]{3}$.

What the Engine loads after compile

Copied to controls.atomicSubcriteria. Coverage and claim mapping treat these as assessment objects under the parent capability.

A1-SC-001 | A1-Q1

Purpose, users, outcomes and use boundaries are explicit and attributable.

required_evidence_ids	EVD-A1-001
minimum_technical_assurance	DECLARED
required_human_assurance	HUMAN_VALIDATED

A1-SC-002 | A1-Q2

Implemented use paths remain within the declared purpose boundary.

required_evidence_ids	EVD-A1-002
minimum_technical_assurance	IMPLEMENTED
required_human_assurance	NOT_REQUIRED

4. Evidence requirements

SCHEMA ENGINE

Canonical JSON

required_evidence[] {id, title, description, minimum_technical_assurance, required_human_assurance}. Evidence IDs match ^EVD-(?:AP-)?[A-F][1-5]-[0-9]{3}$.

What the Engine loads after compile

Copied to controls.requiredEvidence. Schema 2.1.0 does not include evidence class, acceptance conditions or limitations; put those constraints in description and evidence_rules or they stay publication-only.

EVD-A1-001 — Purpose record

Attributable record of intended purpose, users and use boundaries.

minimum_technical_assurance	DECLARED
required_human_assurance	HUMAN_VALIDATED

EVD-A1-002 — Implementation boundary

Technical evidence connecting implemented use paths to the declared purpose.

minimum_technical_assurance	IMPLEMENTED
required_human_assurance	NOT_REQUIRED

capability_indicators

- Approved purpose and use-boundary record
- Named accountable owner

5. Evidence discipline

SCHEMA ENGINE

Canonical JSON

evidence_rules {sufficiency[], evidence_ceiling, false_positive_guards[], prohibited_inferences[]}

What the Engine loads after compile

Copied as controls.evidenceRules. The compiler also flattens falsePositiveGuards and prohibitedInferences onto the control for deterministic assessment.

evidence_ceiling	Repository metadata alone cannot establish accountable ownership or approval.

sufficiency

- Purpose and implemented use paths must both be represented by attributable evidence.

false_positive_guards

- Do not treat a package description as an approved purpose record.

prohibited_inferences

- Do not infer an accountable owner from commit authorship.

6. Findings and gate effects

SCHEMA ENGINE

Canonical JSON

finding_definitions[] {id, assessment_object_id, title, eligible_states, severity, human_decision_authority}; hard_gate_effect {effect, conditions, override_authority}

What the Engine loads after compile

Copied to controls.findingDefinitions and controls.hardGateEffect, and also onto the paired requirement. A locked cognitive finding must reference a published finding ID such as FND-A1-001 before Playbook tactics retrieve.

hard_gate_effect.effect	BLOCK
override_authority	Governance decision authority

- Deployment lacks an accountable owner or bounded purpose.

FND-A1-001 — Purpose or accountable owner is not adequately established

assessment_object_id	A1
eligible_states	PARTIALLY_SATISFIED, NOT_SATISFIED, UNKNOWN
severity	HIGH
human_decision_authority	GOVERNANCE

Richer finding tables in long-form PDFs

Atomic-item lists, linked evidence IDs, lifecycle consequence and human-lock flags appear in some publication PDFs. Schema 2.1.0 findings do not have those properties. Keep extra reviewer text in the PDF; keep Engine-actionable identity in finding_definitions.

8. Lifecycle assurance targets

SCHEMA ENGINE

Canonical JSON

target_assurance_by_lifecycle_stage[] {lifecycle_stage, minimum_technical_assurance, required_human_assurance}

What the Engine loads after compile

Compiler splits these into minimumTechnicalAssuranceByLifecycle, requiredHumanAssuranceByLifecycle and a combined targetStateByLifecycle. Control assessment uses the target for the requested transition.

QUALIFICATION AND REGISTRATION	DECLARED technical / HUMAN_VALIDATED human
DESIGN AND DEVELOPMENT	IMPLEMENTED technical / HUMAN_VALIDATED human
VERIFICATION AND VALIDATION	TESTED technical / HUMAN_VALIDATED human
DEPLOYMENT	TESTED technical / FORMALLY_APPROVED human
OPERATION AND MONITORING	OPERATIONALLY_OBSERVED technical / HUMAN_VALIDATED human

9. Normative source mappings

SCHEMA ENGINE

Canonical JSON

normative_source_mappings[] {source_id, relevant_locator, mapping_rationale, last_verified_on} plus a register entry for that source_id

What the Engine loads after compile

Copied to requirements.normativeMappings. The Engine treats official locators as provenance. They do not decide legal applicability.

source_id	SRC-INTERNAL-GOV-2026
relevant_locator	Qualification and registration
mapping_rationale	Defines the organizational purpose and ownership record.
last_verified_on	2026-07-31

10. Approved tactic mappings

SCHEMA ENGINE

Canonical JSON

candidate_tactic_refs[] {tactic_id, tactic_version, relationship, mapping_status}. tactic_id must match TAC-(PURPOSE|DATA|MODELS|ARCHITECTURE|HUMAN|ACCOUNTABILITY)-(AP-)?[A-F][1-5]-[0-9]{2}

What the Engine loads after compile

Playbook retrieval is driven by locked finding object IDs and the catalog Primary object / mapping, not by free-text similarity. Empty candidate_tactic_refs is valid; the catalog still maps TAC-PURPOSE-A1-01 to A1 / AP-A1.

This sample leaves candidate_tactic_refs empty. Production objects may list approved refs that already exist as Primary object / mapping rows in the Tactic Catalog.

11. Runtime decision boundary

PUBLICATION

Not a schema object

The long-form publication states what the machine may extract, must not decide, and which acts remain human. Encode the enforceable part in evidence_rules and hard_gate_effect. The Engine already separates locked findings, hard gates, publication integrity and formal approval.

Machine may

- Extract and compare purpose statements, system behavior, use records and change evidence.
- Identify missing, conflicting, stale or scope-mismatched evidence.
- Apply deterministic evidence ceilings and propose eligible conclusion states for human lock.
- Retrieve exact approved tactic mappings after a finding is locked.

Machine must not

- Define the organization's purpose or risk appetite.
- Treat legal vocabulary as a legal-applicability conclusion.
- Raise assurance beyond the evidence ceiling.
- Clear the hard gate, accept residual risk, authorize a new use or lifecycle transition.
- Change or close a locked finding because a tactic was selected.

12. Approval record

SCHEMA ENGINE

Canonical JSON

approval_record is null for DRAFT/PILOT. APPROVED/FROZEN require {approved_by_role, approved_on, approval_scope, approved_version}.

What the Engine loads after compile

Production compilation refuses unapproved objects. Engine diagnostics expose knowledgeBaseStatus and releaseStatus; they do not treat this PDF as approval evidence.

release_status	DRAFT
approval_record	null

OBJECT / RUNTIME

AP-A1 — Undefined or elastic intended purpose

Version 1.0.0 | DRAFT | ANTIPATTERN | paired A1

Canonical definition

Purpose is vague, technically framed, inconsistent, or expands without controlled requalification.

Failure mechanism and consequences

SCHEMA ENGINE

Canonical JSON

failure_mechanism, potential_consequences[] (anti-pattern fields; capabilities use governance_purpose / distinct_claim instead)

What the Engine loads after compile

Copied to antipatterns.failureMechanism and antipatterns.consequences.

Ambiguous purpose permits uncontrolled requirements, data use and deployment boundaries.

- Uncontrolled use expansion
- Unclear accountability

3. Atomic tests

SCHEMA ENGINE

Canonical JSON

atomic_tests[] {id, question_id, test, required_evidence_ids, minimum_technical_assurance, required_human_assurance}. IDs match ^AP-[A-F][1-5]-AT-[0-9]{3}$.

What the Engine loads after compile

Copied to antipatterns.atomicTests. These are the anti-pattern assessment objects in the coverage matrix.

AP-A1-AT-001 | AP-A1-Q1

Test whether the purpose record contains users, outcomes and explicit use boundaries.

required_evidence_ids	EVD-AP-A1-001
minimum_technical_assurance	DECLARED
required_human_assurance	HUMAN_VALIDATED

AP-A1-AT-002 | AP-A1-Q2

Compare declared, implemented and operational uses across the assessed boundary.

required_evidence_ids	EVD-AP-A1-002
minimum_technical_assurance	TESTED
required_human_assurance	NOT_REQUIRED

2. Primary questions

AP-A1-Q1	DEFINITION AND INTENT — Is the declared purpose vague, inconsistent or missing material use boundaries?
AP-A1-Q2	IMPLEMENTATION AND OPERATION — Do implemented or operational uses materially diverge from the declared purpose?
AP-A1-Q3	EVIDENCE AND EFFECTIVENESS — Has a scoped current comparison tested declared and actual uses for divergence?

> **Compile note**
>
> Anti-pattern primary_questions are required by schema 2.1.0. The current compiler copies capability questions onto controls.questions; anti-pattern questions remain on the authoring object and should still be written because they are the human and cognitive assessment prompts for AP objects.

4–5. Evidence and discipline

EVD-AP-A1-001 — Purpose record

Attributable declared purpose and explicit use boundaries.

minimum_technical_assurance	DECLARED
required_human_assurance	HUMAN_VALIDATED

EVD-AP-A1-002 — Purpose comparison

Scoped comparison of declared, implemented and operational uses.

minimum_technical_assurance	TESTED
required_human_assurance	NOT_REQUIRED

evidence_ceiling	Silence or a single document cannot establish tested absence of purpose drift.

- Do not infer tested absence from a single README or purpose statement.

6. Findings and gate effects

hard_gate_effect.effect	BLOCK
override_authority	Governance decision authority

- Purpose is undefined or materially inconsistent at deployment.

FND-AP-A1-001 — Purpose is undefined or materially inconsistent

eligible_states	CONFIRMED_PRESENT, PARTIALLY_PRESENT, UNKNOWN
severity	HIGH

7. Tested-absence contract

SCHEMA ENGINE

Canonical JSON

absence_test_contract {scope_defined, executed, successful, current, independently_verified, required_artifacts[]} — all five booleans are const true in schema 2.1.0

What the Engine loads after compile

Copied to antipatterns.absenceTestContract. The Engine must not promote UNKNOWN or missing incidents to TESTED_ABSENT.

scope_defined	true
executed	true
successful	true
current	true
independently_verified	true

- Scoped purpose-to-implementation comparison

OBJECT / RUNTIME

What the Engine actually loads

Authoring JSON is compiled into six runtime collections. Production starts only from a hash-verified manifest of those files. The tables below show the compiled shape for this sample pair — the JSON the Engine uses — not the PDF text.

Compiled requirement excerpt / requirements.json / Engine applicability and interpretation


```
{
  "id": "REQ-A1",
  "authoringObjectId": "A1",
  "interpretation": "Intended purpose, users, affected persons, operating context, outputs, permitted and
prohibited uses, and foreseeable misuse are precise enough to govern design and evaluation.",
  "governancePurpose": "Prevent purpose ambiguity and uncontrolled expansion of the solution use boundary.",
  "authority": "INTERNAL_PROCESS",
  "applicability": "ALWAYS",
  "humanAuthority": "GOVERNANCE",
  "findingDefinitions": [
    {
      "id": "FND-A1-001",
      "assessment_object_id": "A1",
      "title": "Purpose or accountable owner is not adequately established",
      "eligible_states": [
        "PARTIALLY_SATISFIED",
        "NOT_SATISFIED",
        "UNKNOWN"
      ],
      "severity": "HIGH",
      "human_decision_authority": "GOVERNANCE"
    }
  ]
}
```


Compiled control excerpt / controls.json / Engine assessment, coverage, gates, evidence rules


```
{
  "id": "CTRL-A1",
  "authoringObjectId": "A1",
  "pairedObjectId": "AP-A1",
  "severity": "HIGH",
  "signals": [
    "purpose",
    "accountable-owner",
    "excluded-use"
  ],
  "questions": [
    "A1-Q1",
    "A1-Q2",
    "A1-Q3"
  ],
  "atomicSubcriteria": [
    "A1-SC-001",
    "A1-SC-002"
  ],
  "requiredEvidence": [
    "EVD-A1-001",
    "EVD-A1-002"
  ],
  "findingDefinitions": [
    "FND-A1-001"
  ],
  "hardGateEffect": {
    "effect": "BLOCK",
    "conditions": [
      "Deployment lacks an accountable owner or bounded purpose."
    ],
    "override_authority": "Governance decision authority"
  },
  "evidenceRules": {
    "evidence_ceiling": "Repository metadata alone cannot establish accountable ownership or approval.",
    "false_positive_guards": [
```

Compiled anti-pattern excerpt / antipatterns.json / Engine anti-pattern state and absence tests


```
{
  "id": "AP-A1",
  "pairedObjectId": "A1",
  "relatedControlIds": [
    "CTRL-A1"
  ],
  "signal": "undefined-purpose",
  "failureMechanism": "Ambiguous purpose permits uncontrolled requirements, data use and deployment boundaries.",
  "atomicTests": [
    "AP-A1-AT-001",
    "AP-A1-AT-002"
  ],
  "findingDefinitions": [
    "FND-AP-A1-001"
  ],
  "absenceTestContract": {
    "scope_defined": true,
    "executed": true,
    "successful": true,
    "current": true,
    "independently_verified": true,
    "required_artifacts": [
      "Scoped purpose-to-implementation comparison"
    ]
  },
  "hardGateEffect": {
    "effect": "BLOCK",
    "conditions": [
      "Purpose is undefined or materially inconsistent at deployment."
    ],
    "override_authority": "Governance decision authority"
  }
}
```


Field path from human document to Engine

Human heading	Canonical JSON	Runtime collection / Engine use
Canonical definition	canonical_definition	requirements.interpretation
Governance purpose	governance_purpose	requirements.governancePurpose
Distinct claim	distinct_claim	Authoring/review only in current compiler
Signals	runtime_signals	controls.signals / antipatterns.signals
Questions	primary_questions	controls.questions; coverage matrix
Atomic logic	atomic_subcriteria / atomic_tests	controls.atomicSubcriteria / antipatterns.atomicTests
Evidence items	required_evidence	controls.requiredEvidence
Discipline	evidence_rules	controls.evidenceRules and flattened guards
Findings	finding_definitions	findingDefinitions; claim lock IDs
Hard gate	hard_gate_effect	controls.hardGateEffect / antipatterns.hardGateEffect
Lifecycle targets	target_assurance_by_lifecycle_stage	targetStateByLifecycle used by assessControls
Sources	normative_source_mappings	requirements.normativeMappings + normativeSources
Tactics	candidate_tactic_refs + Playbook mapping	selectPlaybookActions after locked findings
Absence contract	absence_test_contract	antipatterns.absenceTestContract

Approval	approval_record + release_status	Compile gate and knowledge diagnostics

Do not send the PDF to the Engine

- The Engine knowledge provider loads hash-verified JSON collections, never category PDFs.
- Human PDFs may be stored beside the Blob objects for reviewers; they are absent from the runtime manifest.
- If a sentence must change a gate, finding, evidence ceiling or tactic mapping, put it in canonical JSON first, then regenerate the PDF.
- Production activation is a separate Engine step after the Maintainer publishes an APPROVED manifest.